QUICK_START_TOMORROW

Quick Start Guide for Tomorrow

☕ **Date:** 2025-11-05 (Tomorrow) **Phase:** Phase 1, Day 1 **Goal:** Create mock server infrastructure for testing

---

Morning Setup (15 minutes)

1. Review Yesterday's Work

Check what was completed:

```
cd /home/milosvasic/Projects/HelixDevelopment/HelixCode

# Run existing tests to verify everything still works
cd HelixCode
go test ./internal/notification/... -v

# Should see:
# - All Slack tests passing
# - All Telegram tests passing
# - All Email tests passing
# PASS
```

2. Read the Roadmap

🚀 Open and review: - `NOTIFICATION_INTEGRATION_REPORT.md` - Overall analysis - `IMPLEMENTATION_ROADMAP.md` - Detailed plan - Focus on **Phase 1, Day 1-2** section

---

Today's Tasks (Phase 1, Day 1-2)

Task 1: Create Mock Server Directory

```
cd /home/milosvasic/Projects/HelixDevelopment/helix_code/
        HelixCode
mkdir -p internal/notification/testutil
cd internal/notification/testutil
```

Task 2: Implement Mock Slack Server

File to create: `mock_servers.go`

Copy this code:

```
package testutil

import (
    "encoding/json"
    "io"
    "net/http"
    "net/http/httptest"
    "sync"
)

// MockSlackServer simulates Slack webhook endpoint
type MockSlackServer struct {
    *httptest.Server
    Requests []SlackRequest
    mutex    sync.Mutex
}

type SlackRequest struct {
    Channel   string `json:"channel"`
    Username  string `json:"username"`
    Text      string `json:"text"`
    IconEmoji string `json:"icon_emoji"`
}

func NewMockSlackServer() *MockSlackServer {
    mock := &MockSlackServer{
        Requests: make([]SlackRequest, 0),
    }

    mock.Server = httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
        mock.mutex.Lock()
        defer mock.mutex.Unlock()

        // Verify it's POST
        if r.Method != http.MethodPost {
            w.WriteHeader(http.StatusMethodNotAllowed)
            return
        }
```

```
        // Parse request
        var req SlackRequest
        body, _ := io.ReadAll(r.Body)
        if err := json.Unmarshal(body, &req); err != nil {
            w.WriteHeader(http.StatusBadRequest)
            return
        }

        mock.Requests = append(mock.Requests, req)
        w.WriteHeader(http.StatusOK)
    }))

    return mock
}

func (m *MockSlackServer) GetRequests() []SlackRequest {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    return append([]SlackRequest{}, m.Requests...)
}

func (m *MockSlackServer) Reset() {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    m.Requests = make([]SlackRequest, 0)
}

func (m *MockSlackServer) GetRequestCount() int {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    return len(m.Requests)
}
```

Save the file.

Task 3: Test the Mock Server

Create test file: `mock_servers_test.go`

```
package testutil

import (
    "bytes"
    "encoding/json"
```

```
    "net/http"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestMockSlackServer(t *testing.T) {
    server := NewMockSlackServer()
    defer server.Close()

    // Send a test request
    payload := SlackRequest{
        Channel:   "#test",
        Username:  "testbot",
        Text:      "Hello",
        IconEmoji: ":rocket:",
    }

    jsonData, _ := json.Marshal(payload)
    resp, err := http.Post(server.URL, "application/json",
        bytes.NewReader(jsonData))
    require.NoError(t, err)
    assert.Equal(t, http.StatusOK, resp.StatusCode)

    // Verify request was captured
    requests := server.GetRequests()
    assert.Equal(t, 1, len(requests))
    assert.Equal(t, "#test", requests[0].Channel)
    assert.Equal(t, "testbot", requests[0].Username)
}

func TestMockSlackServer_Reset(t *testing.T) {
    server := NewMockSlackServer()
    defer server.Close()

    // Send request
    payload := SlackRequest{Channel: "#test"}
    jsonData, _ := json.Marshal(payload)
    http.Post(server.URL, "application/json",
        bytes.NewReader(jsonData))

    assert.Equal(t, 1, server.GetRequestCount())
```

```
    // Reset
    server.Reset()
    assert.Equal(t, 0, server.GetRequestCount())
}
```

Run the test:

```
cd /home/milosvasic/Projects/HelixDevelopment/helix_code/
        HelixCode
go test ./internal/notification/testutil/... -v
```

Expected output:

```
=== RUN   TestMockSlackServer
--- PASS: TestMockSlackServer
=== RUN   TestMockSlackServer_Reset
--- PASS: TestMockSlackServer_Reset
PASS
```

Checkpoint: If tests pass, you've successfully created mock Slack server!

Task 4: Add Mock Telegram Server

Add to `mock_servers.go`:

```
// MockTelegramServer simulates Telegram Bot API
type MockTelegramServer struct {
    *httptest.Server
    Requests []TelegramRequest
    mutex    sync.Mutex
}

type TelegramRequest struct {
    ChatID    string `json:"chat_id"`
    Text      string `json:"text"`
    ParseMode string `json:"parse_mode"`
}

func NewMockTelegramServer() *MockTelegramServer {
    mock := &MockTelegramServer{
        Requests: make([]TelegramRequest, 0),
    }

    mock.Server = httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
```

```
        mock.mutex.Lock()
        defer mock.mutex.Unlock()

        if r.Method != http.MethodPost {
            w.WriteHeader(http.StatusMethodNotAllowed)
            return
        }

        var req TelegramRequest
        body, _ := io.ReadAll(r.Body)
        if err := json.Unmarshal(body, &req); err != nil {
            w.WriteHeader(http.StatusBadRequest)
            return
        }

        mock.Requests = append(mock.Requests, req)

        // Return successful Telegram API response
        response := map[string]interface{}{
            "ok": true,
            "result": map[string]interface{}{
                "message_id": len(mock.Requests),
            },
        }

        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(response)
    }))

    return mock
}

func (m *MockTelegramServer) GetRequests() []TelegramRequest {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    return append([]TelegramRequest{}, m.Requests...)
}

func (m *MockTelegramServer) Reset() {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    m.Requests = make([]TelegramRequest, 0)
}
```

Add test for Telegram server in `mock_servers_test.go`

Run tests again:

```
go test ./internal/notification/testutil/... -v
```

All should pass!

Task 5: Add Mock Discord Server

Add to `mock_servers.go`:

```go
// MockDiscordServer simulates Discord webhook endpoint
type MockDiscordServer struct {
    *httptest.Server
    Requests []DiscordRequest
    mutex    sync.Mutex
}

type DiscordRequest struct {
    Content string `json:"content"`
}

func NewMockDiscordServer() *MockDiscordServer {
    mock := &MockDiscordServer{
        Requests: make([]DiscordRequest, 0),
    }

    mock.Server = httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
        mock.mutex.Lock()
        defer mock.mutex.Unlock()

        if r.Method != http.MethodPost {
            w.WriteHeader(http.StatusMethodNotAllowed)
            return
        }

        var req DiscordRequest
        body, _ := io.ReadAll(r.Body)
        if err := json.Unmarshal(body, &req); err != nil {
            w.WriteHeader(http.StatusBadRequest)
            return
        }
```

```
        mock.Requests = append(mock.Requests, req)
        w.WriteHeader(http.StatusNoContent)
    }))

    return mock
}

func (m *MockDiscordServer) GetRequests() []DiscordRequest {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    return append([]DiscordRequest{}, m.Requests...)
}

func (m *MockDiscordServer) Reset() {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    m.Requests = make([]DiscordRequest, 0)
}
```

Add test, run again.

End of Day Checklist

Before Committing:

1. **Run all tests:**

```
cd /home/milosvasic/Projects/HelixDevelopment/helix_code/
        HelixCode

# Test mock servers
go test ./internal/notification/testutil/... -v

# Test all notification code
go test ./internal/notification/... -v

# Should see ALL PASS
```

1. **Check code quality:**

```
# Run linter
go vet ./internal/notification/...
```

```
# Format code
go fmt ./internal/notification/...
```

1. **Update progress:**
    - Mark tasks as complete in `IMPLEMENTATION_ROADMAP.md`
    - Note any blockers or issues
2. **Commit your work:**

```
git add .
git commit -m
        "feat(notification): Add mock server infrastructure for
        testing

- Implemented MockSlackServer with request capture
- Implemented MockTelegramServer with API response simulation
- Implemented MockDiscordServer
- Added comprehensive tests for all mock servers
- All tests passing

Part of Phase 1: Mock Servers & Testing Infrastructure
Ref: IMPLEMENTATION_ROADMAP.md Phase 1, Day 1-2"
```

Success Criteria for Today

By end of day, you should have:

- `internal/notification/testutil/mock_servers.go` file created
- `internal/notification/testutil/mock_servers_test.go` file created
- MockSlackServer implemented and tested
- MockTelegramServer implemented and tested
- MockDiscordServer implemented and tested
- All tests passing
- Code committed to git

Tomorrow (Day 3)

Next tasks: 1. Create integration tests using mock servers 2. Test Slack integration with mock server 3. Test Telegram integration with mock server 4. Test Discord integration with mock server

File to create: `helix_code/test/integration/slack_integration_test.go`

📚

See `IMPLEMENTATION_ROADMAP.md` Phase 1, Day 3-4 for details.

Reference Documents

Keep these open while working:

1. `IMPLEMENTATION_ROADMAP.md` - Detailed implementation plan
2. `NOTIFICATION_INTEGRATION_REPORT.md` - Overall analysis and research
3. `INTEGRATION_IMPLEMENTATION_SUMMARY.md` - What's been done so far

Code reference: - `internal/notification/engine.go` - Main notification 🆘 engine - `internal/notification/slack_test.go` - Example tests - `internal/notification/telegram_test.go` - Example tests

---

If You Get Stuck

1. **Tests failing?**
   - Check error messages carefully
   - Run single test: `go test -run TestName -v`
   - Add debug prints: `t.Logf("Debug: %v", variable)`
2. **Import errors?**
   - Run: `go mod tidy`
   - Check import paths match directory structure
3. **Mock server not working?**
   - Check the server is started: `server := NewMockSlackServer()`
   - Check you're using `server.URL` in requests
   - Check you're calling `defer server.Close()`
4. **Need help?**
   - Review existing test files for examples
   - Check Go testing docs: https://pkg.go.dev/testing
   - Check httptest docs: https://pkg.go.dev/net/http/httptest

💪

Motivation

What you accomplished yesterday: - Fixed critical email bug - Implemented complete Telegram integration - Created 100% test coverage for all channels - Created comprehensive documentation

You're on track! Keep up the great work!

Today's goal: Build the testing infrastructure that will ensure reliability for all future integrations.

Good luck! You've got this!